DEPARTMENT OF SOFTWARE ENGINEERING
Faculty of Computing & Information Technology (FCIT)
University of the Punjab, Lahore – PAKISTAN



MedBill D1 Agile Report

Product Backlog and Definition of Done

BSEF23-51 MedBill

Bachelors of Science in Software Engineering (2023-2027)

By

Hasan Abdullah BSEF23M552

Irfan Saleem BSEF23M553

Supervised by:

Prof. Omair Babar
Primary Supervisor

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY, UNIVERSITY OF THE PUNJAB, LAHORE.



Contents

1. Product Overview....2
2. System Design High Level....2
3. Features and Scope....3
4. Product Backlog....4
5. Prototype Demonstration....5
6. Progress Snapshot....7
7. Definition of Done....7
8. Development Approach....8
9. CI CD Plan Brief....8
10. Next Steps Semester 8....9

1. Product Overview

MedBill is a web-based medical billing and revenue-cycle system for small clinics and billing teams. The D1 rebuild establishes a working patient-to-claim flow and a tested architecture that can grow into payment posting, denial management, reporting, CMS-1500 export, and AI-assisted risk checking.

Problem. Small teams often split patient, claim, and payment work across spreadsheets or disconnected tools. That fragmentation makes missing insurance details and coding mistakes harder to detect before a claim is prepared.

Target users. The primary users are Billers who prepare and review claims and Administrators who control users and master data. The D1 demo uses synthetic records only.

D1 solution. The system stores structured patient and insurance data, builds claim drafts, and runs explainable backend rules. A draft either becomes Ready or remains Validation failed with messages the Biller can act on.

2. System Design High Level

MedBill uses a three-layer web architecture. React renders the interface, Django REST Framework exposes authenticated JSON APIs and owns business rules, and a relational database stores the records. The browser cannot bypass validation by changing a screen because claim checks execute in Django.

React browser interface	Django REST API	Relational database
Forms, worklists, dashboard	Authentication, validation, calculations	Patients, claims, audit-ready records

Technology choices

- Frontend: React 18, Bootstrap 5, Vite 8, and the browser Fetch API.
- Backend: Python 3.12, Django 5.2, and Django REST Framework 3.18.
- Database: SQLite for a reliable zero-setup D1 demo; MySQL remains the proposal-aligned deployment target.
- Verification: Django automated tests, Playwright browser tests, production build checks, and npm dependency audit.
- Deployment direction: Docker Compose and GitHub Actions after the D1 scope is accepted.

3. Features and Scope

Core epics

- Access and accountability
- Patient and insurance management
- Charge entry and claim preparation
- Rule-based and AI-assisted claim verification
- CMS-1500 preview and PDF export
- Payment, denial, and claim status management
- Dashboard, A R aging, revenue, and denial reports

D1 implementation boundary

Implemented	Partially implemented	Planned for Semester 8
Admin and Biller login	Role enforcement uses the Admin staff flag; user-management screens are not yet included	Full user and role administration
Patient and insurance capture	Editing, duplicate checks, search, and archive rules remain	Master data and audit expansion
Claim draft and four server-side validation checks	The lifecycle currently stops at Ready or Validation failed	Submission, payment, denial, and closing states
Dashboard counts and draft billed value	Only D1 metrics are calculated	A R aging, revenue, denial, and follow-up reports
Automated API and browser tests	CI configuration is documented but not yet hosted	GitHub Actions and deployment checks
		CMS-1500 PDF and evaluated ML risk model

Direct payer submission, EDI 837 integration, live eligibility checking, payment gateways, mobile applications, autonomous coding, and production compliance certification are outside the approved D1 scope.

4. Product Backlog

The backlog uses Must, Should, and Could priorities. A story enters implementation only after its acceptance check is observable.

ID	Priority	User story and acceptance check	D1 status
AUTH-01	Must	As a user, I can sign in and sign out. Valid credentials return a server token and invalid credentials are rejected.	Done
AUTH-02	Must	As an Admin, I can manage user roles. A Biller cannot perform Admin-only work.	Partial
PAT-01	Must	As a Biller, I can register a patient with insurer and member ID. The record persists and appears in the registry.	Done
PAT-02	Must	As a Biller, I can search, edit, and archive patients without losing history.	Planned
CLM-01	Must	As a Biller, I can create a claim draft for a patient with service date, diagnosis, procedure, and amount.	Done
CLM-02	Must	As a Biller, I receive actionable validation messages for missing coverage, invalid code shape, or non-positive amount.	Done
CLM-03	Must	As a Biller, I can preview and export a CMS-1500-style PDF from a validated claim.	Planned
PAY-01	Must	As a Biller, I can post full or partial payments and keep outstanding balances correct.	Planned
DEN-01	Must	As a Biller, I can record a denial reason and follow-up date.	Planned
RPT-01	Must	As a manager, I can view reconciled A R aging, revenue, and denial reports.	Planned
AI-01	Should	As a Biller, I see an explainable claim-risk flag only if an evaluated model meets the agreed baseline.	Experiment planned
OPS-01	Should	As the team, we run builds and tests automatically on every protected-branch pull request.	Local checks done

5. Prototype Demonstration

The demonstration follows one Biller session. The Biller signs in, reviews the dashboard, creates a claim for the seeded synthetic patient, and receives either Ready or Validation failed from the backend. The second browser test deliberately submits bad codes and a zero amount to prove that the correction path works.

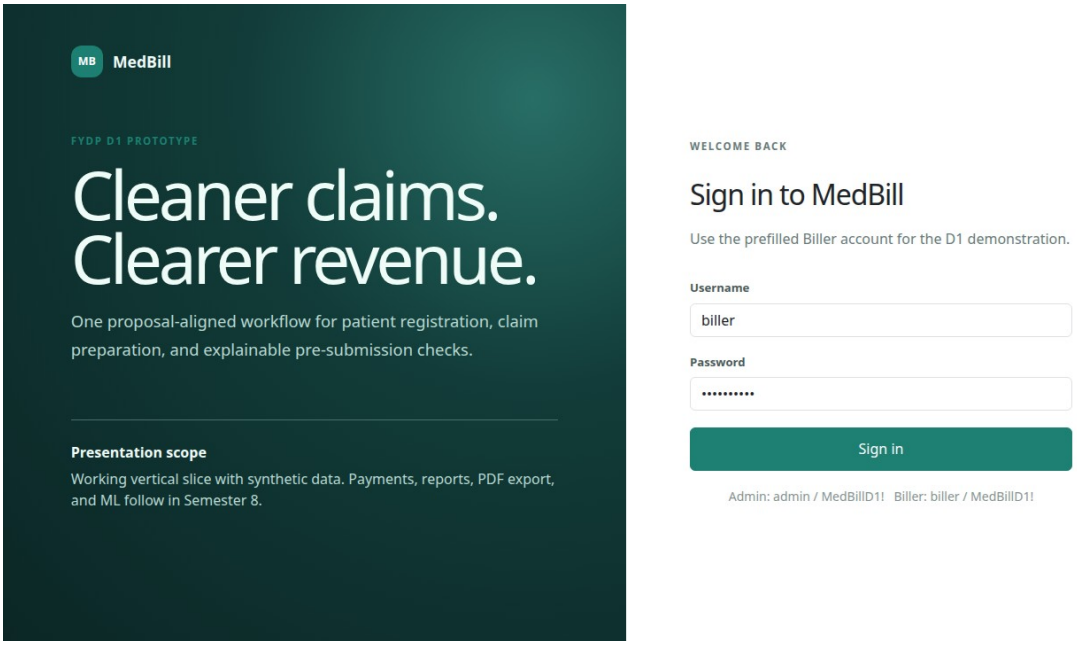


Figure 1 Role-based login and explicit D1 boundary

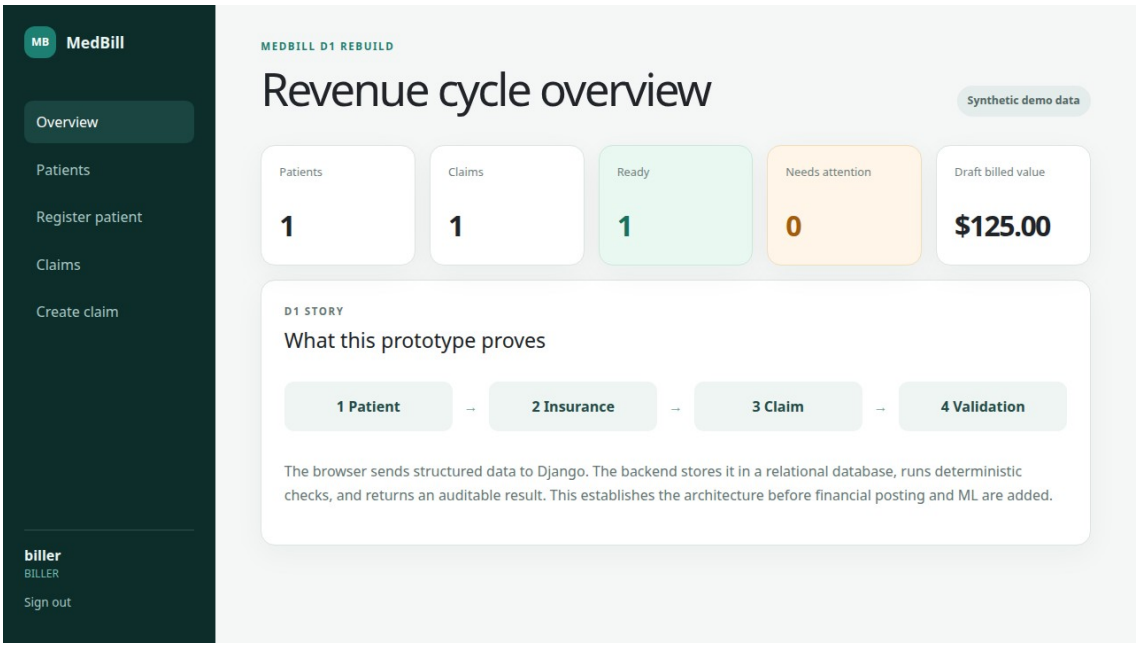


Figure 2 Dashboard after loading the seeded relational records

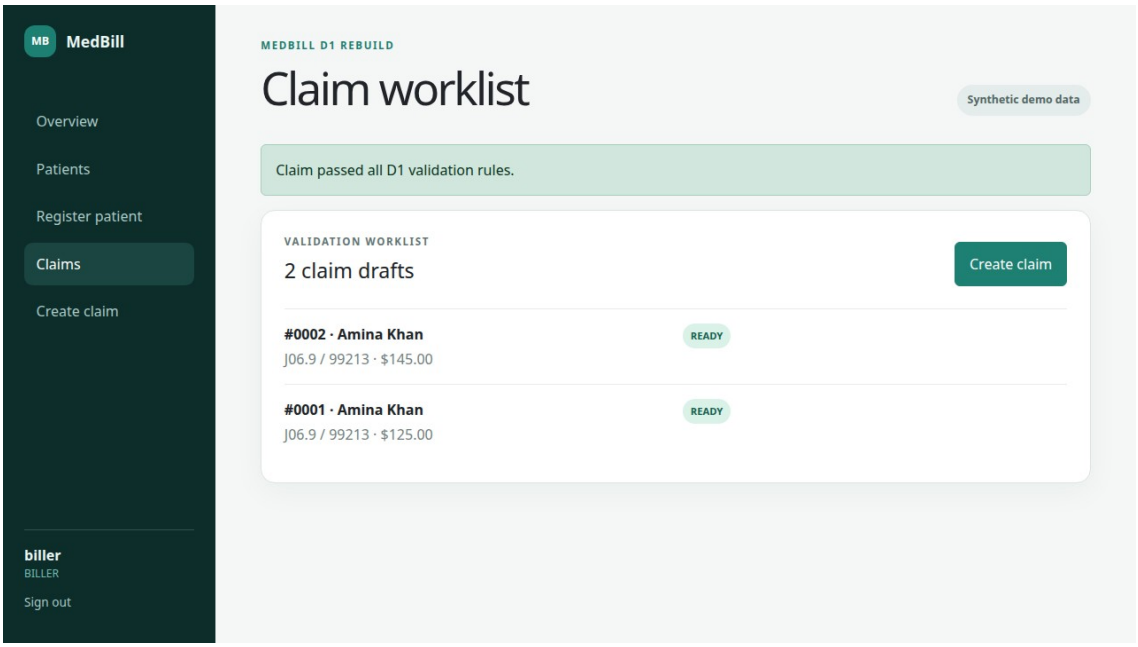


Figure 3 Claim worklist after a draft passes all current validation rules

Current limitations

- The local demo uses SQLite so it runs without a database server. MySQL configuration is present but has not yet been used for this clean rebuild.
- The D1 lifecycle stops after validation. CMS-1500 export, payment posting, denials, and financial reports remain planned.
- The AI component is not claimed as implemented. It requires an approved dataset, baseline comparison, leakage controls, and measured results.

6. Progress Snapshot

A new repository was initialized for the rebuild with a stable main branch and traceable baseline and documentation commits. A remote GitHub URL has not been published yet and should be added before submission.

Evidence	Result	Meaning
Django system check	Passed	No configuration errors
Migration drift check	Passed	Models and migration agree
Backend automated tests	4 of 4 passed	Login support, public health check, valid and invalid claim paths, and dashboard count
Playwright browser tests	2 of 2 passed	Happy path and actionable validation failure path
Frontend production build	Passed	React bundle compiles for deployment
npm dependency audit	0 known vulnerabilities	No currently reported package advisories

The application runs locally with synthetic data. The README contains setup, demo accounts, architecture, scope, and verification commands.

7. Definition of Done

A backlog item is Done only when all applicable checks below pass. Visual presence alone is not sufficient.

- Acceptance criteria are written and linked to a backlog ID.
- Authorization and validation execute in the backend where applicable.
- The complete UI, API, and database path works with synthetic test data.
- Automated tests cover the happy path and the main failure path.
- The production frontend build succeeds without a runtime-blocking error.
- No secrets, real patient data, generated databases, or environment files are committed.
- A second team member reviews the change before it reaches main.
- README, backlog status, and D1 evidence match the demonstrated software.

8. Development Approach

The team will use two-week Scrum sprints. Each sprint begins with a small goal, implements one or more vertical slices, and ends with a review, retrospective, tested increment, and documentation update.

Branching strategy

- main contains stable, demonstrable releases.
- develop contains integrated work for the active sprint.
- feature or fix branches remain short-lived and map to backlog IDs.

Change workflow

Refine the story, create a branch, implement the complete path, run checks, open a pull request, review the evidence, merge to develop, demonstrate at sprint review, then promote a stable increment to main.

Team responsibilities

- Hasan Abdullah: Scrum Master, backend and AI lead.
- Irfan Saleem: Product Owner, frontend and database lead.
- Both students: requirements, acceptance criteria, code review, testing, demonstrations, and documentation.

9. CI CD Plan Brief

GitHub Actions will run on pull requests to develop and main. A change may merge only when the required checks pass.

Stage	Automated check	Failure response
Backend	Install Python dependencies, system check, migration drift check, and tests	Block merge
Frontend	npm clean install, dependency audit, and production build	Block merge
Browser	Playwright happy and failure paths against seeded synthetic data	Block release
Packaging	Docker Compose configuration and health checks	Block deployment

Deployment will be added after the database, secrets, HTTPS, backup, and recovery settings are reviewed. Local demonstration remains the fallback for evaluations.

10. Next Steps Semester 8

Target	Planned outcome
Sprint 1	Complete Admin user management, master data, patient editing, search, duplicate handling, and audit events.
Sprint 2	Add charge lines, documented claim-state transitions, and CMS-1500 preview and PDF export.
Sprint 3	Implement payment, partial payment, adjustment, denial, follow-up, and outstanding-balance rules.
Sprint 4	Build A R aging, revenue, denial, and dashboard reports and reconcile them against fixed examples.
Sprint 5	Run the approved ML experiment, compare it with rules-only checks, document metrics, and integrate only if the agreed baseline is met.
Release	Add MySQL and Docker deployment, GitHub Actions, security checks, backup and restore rehearsal, user documentation, and UAT evidence.

The immediate presentation goal is to demonstrate the tested D1 flow, explain the three-layer architecture, show why backend validation matters, and state the remaining scope without overstating completion.